



Instituto Superior Politécnico de Tecnologias e Ciências
(ISPTEC)

Departamento de Engenharia e Tecnologias

Base de Dados II

Licenciatura em Engenharia Informática

2025/2026

Lab #02 — Exploração de Banco de Dados Distribuídos

Judson Paiva

judson.paiva@isptec.co.ao

Ficha Técnica e Calendário

Grupos	6 grupos de 3 a 4 elementos — líderes eleitos por grupo
Uso de IA	Permitido e obrigatório de registar — prompts, ferramentas usadas e correcções feitas
Dataset	MercadoKwanza — desenvolvido pelos líderes dos 6 grupos em conjunto com apoio de IA
GitHub	Todo o trabalho (scripts, queries, notas) deve ser publicado num repositório público antes da defesa
Entrega	Relatório enviado por e-mail para judson.paiva@isptec.co.ao com link do GitHub

Fase	Período	O que é entregue
PARTE 1 — Exploração	8 a 14 de Junho de 2026	Relatório com resultados e reflexões da Parte 1
PARTE 2 — Defesa	15 a 20 de Junho de 2026	Defesa pública + Relatório completo (Parte 1 + 2) + link GitHub

Como registar o uso de IA neste laboratório

Sempre que usarem uma ferramenta de IA (ChatGPT, Claude, Gemini, Copilot, etc.), devem registar no relatório:

- Ferramenta usada (ex.: Claude Sonnet).
- Prompt submetido — o que foi perguntado, em resumo.
- O que a IA respondeu — em resumo.
- O que o grupo corrigiu, melhorou ou rejeitou da resposta da IA.

As conclusões e reflexões escritas são SEMPRE individuais e com as próprias palavras.

Copiar texto da IA para as reflexões implica penalização directa.

Pré-Requisitos — Leituras e Pesquisas Antes de Começar

Antes de iniciar qualquer instalação ou execução, o grupo deve pesquisar e compreender os conceitos abaixo. Podem usar a IA, o YouTube, os livros indicados ou artigos da web — o importante é chegar ao laboratório com uma ideia clara de cada um.

Conceito a pesquisar	Questão orientadora — o que devem conseguir responder
----------------------	---

O que é um Banco de Dados Distribuído?	Qual a diferença entre um BD centralizado e um distribuído? Dê um exemplo do dia-a-dia.
O que é o Docker?	Para que serve um contentor? Qual a diferença entre um contentor e uma máquina virtual?
O que é a Replicação?	O que significa Master e Slave numa arquitectura de replicação? O que acontece se o Master cair?
O que é a Fragmentação?	Qual a diferença entre fragmentação horizontal e vertical? Dê um exemplo com uma tabela de clientes.
O que é uma Transação?	O que significam as propriedades ACID? Por que é importante o ROLLBACK?
O que é o Teorema CAP?	Um sistema distribuído pode garantir simultaneamente Consistência, Disponibilidade e Tolerância a Partições? Porquê?
SQL vs NoSQL	Em que situações o MongoDB é preferível ao MySQL? E o MySQL ao MongoDB?
O que é o GitHub?	Como criar um repositório, fazer commit e push de ficheiros? O que é um README.md?

Fontes sugeridas para pesquisa

- KLEPPMANN, M. (2017). Designing Data-Intensive Applications. O'Reilly. — Capítulos 5, 6 e 9.
- ÖZSU, M. T.; VALDURIEZ, P. (2020). Principles of Distributed Database Systems. 4.ª ed. Springer. — Capítulos 1, 3 e 5.
- YouTube: 'Docker in 100 Seconds' (Fireship) — introdução rápida ao Docker.
- YouTube: 'MySQL Replication Tutorial' (TechWorld with Nana) — replicação Master-Slave.
- Artigo: Brewer (2000), 'Towards Robust Distributed Systems' — Teorema CAP (pesquisar no Google Scholar).
- Documentação oficial Docker: <https://docs.docker.com>
- Documentação oficial MySQL 8.0: <https://dev.mysql.com/doc/refman/8.0/en/>

PARTE 1 — EXPLORAÇÃO GUIADA

8 a 14 de Junho de 2026 | Todos os grupos fazem esta parte em simultâneo

Fase 0 — Criação do Dataset MercadoKwanza

Antes de começar qualquer instalação, os líderes dos 6 grupos reúnem-se e desenvolvem em conjunto, com apoio de IA, o script SQL que cria e popula o dataset. Este é o único momento de trabalho conjunto entre grupos diferentes. O resultado é um ficheiro único: mercadokwanza.sql, que todos irão usar.

0.1 — Porquê um Dataset Comum?

Ter todos os grupos a trabalhar sobre os mesmos dados permite:

- Comparar resultados durante a defesa — se um grupo obteve X vendas e outro obteve Y sobre os mesmos dados, há algo a explicar.
- Tornar as críticas entre grupos mais fundamentadas — o Grupo Observador conhece exactamente os dados que estão a ser usados.
- Simular um ambiente real de empresa — todos acedem à mesma base de informação.

0.2 — Esquema do Dataset

O MercadoKwanza simula uma empresa de distribuição e retalho angolana com operações em três províncias. As tabelas são:

Tabela	Atributos principais	Descrição e volume
PROVINCIA	id, nome, capital, populacao	3 registos: Luanda, Benguela, Huambo
LOJA	id, nome, provincia_id, morada, activa	15 lojas — 5 por província
PRODUTO	id, descricao, categoria, preco, activo	200 produtos em 5 categorias
STOCK	id, produto_id, loja_id, quantidade, atualizado_em	$200 \times 15 = 3\,000$ registos de stock
CLIENTE	id, nome, nif, provincia_id, telefone, activo	5 000 clientes distribuídos pelas províncias
VENDA	id, loja_id, cliente_id, data_venda, total	~30 000 vendas entre 2023-2024
ITEM_VENDA	id, venda_id, produto_id, qtd, preco_unit, desconto	~80 000 linhas (2-4 itens por venda)

0.3 — Prompt para Gerar o Script com IA

Os líderes devem colar o seguinte prompt numa ferramenta de IA à escolha:

PROMPT A USAR COM A IA — Gerar mercadokwanza.sql

Gera um script SQL completo para MySQL 8.0 com o seguinte:

1. Cria um schema chamado mercadokwanza.
2. Cria as tabelas PROVINCIA, LOJA, PRODUTO, STOCK, CLIENTE, VENDA e ITEM_VENDA com os atributos indicados, incluindo chaves primárias com AUTO_INCREMENT, chaves estrangeiras e índices nas colunas mais usadas em filtros.
3. Usa stored procedures com loops WHILE para inserir: 3 províncias, 15 lojas, 200 produtos (5 categorias: Alimentação, Higiene, Electrónica, Vestuário, Ferragens), 5000 clientes, ~30 000 vendas e os respectivos itens.
4. Os dados devem ter nomes angolanos realistas (nomes de pessoas, moradas de Luanda/Benguela/Huambo).
5. O script deve ser executável directamente com SOURCE no MySQL sem erros.

Após obter o script, o grupo deve testá-lo, corrigir qualquer erro e documentar as correcções no relatório.

REGISTO DE RESULTADOS — FASE-0

N.º de linhas em VENDA após importação	30000
N.º de linhas em ITEM_VENDA	90000
N.º de clientes importados	5000
N.º de produtos importados	3000
Erros encontrados no script da IA (s/n — quantos?)	Sem erros

REFLEXÃO ESCRITA — resposta individual no relatório

1. O script gerado pela IA funcionou directamente ou precisou de correcções? Que tipo de erros surgiram e o que aprenderam ao corrigi-los?

2. Porque é que o grupo decidiu usar stored procedures com loops em vez de INSERT linha a linha? Qual a vantagem em termos de tempo e manutenção?

Fase 1 — O Docker e a Ideia de Nó Distribuído

1.1 — Porquê o Docker?

Imaginem que têm de simular três servidores de bases de dados em computadores diferentes — um em Luanda, um em Benguela, um em Huambo. Na vida real, isso exigiria três máquinas físicas, três instalações do MySQL e uma rede entre elas. Com Docker, conseguem fazer exactamente isso num único computador de laboratório.

Um contentor Docker é como uma caixa selada que contém tudo o que uma aplicação precisa para funcionar: o sistema operativo mínimo, as bibliotecas e o programa em si. Ao contrário de uma máquina virtual, um contentor não virtualiza o hardware — partilha o kernel do sistema operativo anfitrião, o que o torna muito mais leve e rápido de iniciar.

Analogia — Contentor vs Máquina Virtual

Máquina virtual é como alugar um apartamento completo com cozinha, sala e quartos — tem tudo mas ocupa muito espaço e demora a mobilar.

Contentor Docker é como alugar um quarto num hostel — tem exactamente o que precisas, partilha algumas infra-estruturas comuns (cozinha, casa de banho) e é muito mais rápido e barato de montar.

1.2 — Instalação do Docker

6. Aceder a <https://www.docker.com/products/docker-desktop> e descarregar a versão para o vosso sistema operativo.
7. Instalar seguindo o assistente. No Windows, activar o WSL 2 quando solicitado.
8. Verificar a instalação:

```
docker --version
# Resultado esperado: Docker version 26.x.x

docker compose version
# Resultado esperado: Docker Compose version 2.x.x
```

9. Se aparecer um erro de permissão no Linux, executar:

```
sudo usermod -aG docker $USER #
Fechar e reabrir o terminal
```

REGISTO DE RESULTADOS — FASE-1A

Versão do Docker instalada	29.5.2
Versão do Docker Compose	V5.1.3
Sistema Operativo usado pelo grupo	Windows 11

1.3 — Criar a Estrutura de Pastas do Projecto

Criar a seguinte estrutura no computador. Esta organização é importante — cada pasta tem um propósito claro.


```

    image: mysql:8.0      container_name: no-huambo
restart: unless-stopped  environment:
    MYSQL_ROOT_PASSWORD:          kwanza2024
MYSQL_DATABASE: mercadokwanza  ports:
-   '3308:3306'      command: >
    --server-id=3
    --character-set-server=utf8mb4

# — NÓ 4: MONGODB — Sistema NoSQL (usado na Parte 2) — mongodb:
    image: mongo:7.0      container_name: mongo-kwanza
restart: unless-stopped  ports:
-   '27017:27017'      environment:
    MONGO_INITDB_DATABASE: mercadokwanza

```

1.5 — Levantar o Cluster

10. Abrir um terminal na pasta mercadokwanza/ e executar:

```

docker compose up -d
# O -d significa 'detached' — corre em segundo plano
# Na primeira vez, o Docker vai descarregar as imagens (~500MB) — é normal
demorar

```

11. Verificar que os 4 contentores estão activos:

```

docker ps
# Devem aparecer 4 linhas com STATUS = Up
# no-luanda | no-benguela | no-huambo | mongo-kwanza

```

12. Aceder ao nó Luanda e verificar os dados:

```

docker exec -it no-luanda mysql -uroot -pkwanza2024

-- Dentro do MySQL: SHOW
DATABASES;
USE mercadokwanza;
SHOW TABLES;
SELECT COUNT(*) FROM VENDA;
SELECT COUNT(*) FROM CLIENTE; EXIT;

```

REGISTO DE RESULTADOS — FASE-1B

N.º de contentores com STATUS Up	4
Tempo de arranque do cluster (aprox.)	3.8 s

Resultado de COUNT(*) FROM VENDA (Luanda)	30000
Resultado de COUNT(*) FROM CLIENTE (Luanda)	5000
Os dados do SQL foram carregados automaticamente? (S/N)	S

REFLEXÃO ESCRITA — resposta individual no relatório

1. O que significa o mapeamento de portas '3307:3306'? Porque é que os nós Benguela e Huambo não podem também usar a porta 3306 no anfitrião?

R% **Carlos**

O mapeamento 3307:3306 significa que a porta **3307 do computador anfitrião** está ligada à porta **3306 do contentor Docker**, que é a porta padrão do MySQL. Dessa forma, quando alguém acede à porta 3307 no computador, o pedido é encaminhado para o MySQL que está a correr dentro do contentor. Os nós Benguela e Huambo não podem usar a mesma porta 3306 no anfitrião porque uma porta só pode ser utilizada por um processo de cada vez.

José

A expressão 3307:3306 indica que o Docker está a redirecionar o tráfego da porta 3307 do sistema operativo para a porta 3306 do MySQL dentro do contentor. Como existem vários contentores MySQL no mesmo computador, cada um precisa de uma porta diferente no anfitrião. Caso todos utilizassem a porta 3306, ocorreria um conflito e alguns contentores não conseguiriam iniciar.

Emanuel

O primeiro número representa a porta do computador anfitrião e o segundo representa a porta do serviço dentro do contentor. Assim, 3307:3306 permite aceder ao MySQL do contentor através da porta 3307. Os nós Benguela e Huambo não podem usar a mesma porta 3306 do anfitrião porque o sistema não permite que múltiplos serviços escutem simultaneamente na mesma porta.

Líria

O mapeamento 3307:3306 funciona como uma ponte entre o computador e o contentor. A porta 3307 fica disponível no computador e encaminha os pedidos para a porta 3306 do MySQL dentro do contentor. Como os três servidores estão a ser executados no mesmo anfitrião, cada um precisa de uma porta diferente para evitar conflitos de comunicação.

2. Qual a vantagem de usar o volume './dados:/docker-entrypoint-initdb.d'? O que aconteceria se não usassem esse volume e quisessem carregar os dados manualmente?

R%

Carlos

A principal vantagem é a automatização da inicialização da base de dados. Sempre que o contentor é criado pela primeira vez, os ficheiros SQL presentes na pasta dados são executados automaticamente. Sem esse volume, seria necessário entrar no contentor e importar os ficheiros SQL manualmente, tornando o processo mais demorado e sujeito a erros.

José

O volume permite que os scripts de criação e inserção de dados sejam carregados automaticamente pelo MySQL durante a inicialização. Isso facilita bastante a configuração do ambiente. Caso não utilizássemos esse volume, teríamos de executar comandos de importação manualmente sempre que fosse necessário preparar uma nova instância da base de dados.

Emanuel

Ao utilizar o volume, os ficheiros SQL ficam acessíveis ao contentor e podem ser executados sem intervenção do utilizador. Isso garante rapidez e consistência na criação da base de dados. Sem o volume, seria preciso importar os dados através do terminal ou de uma ferramenta gráfica, aumentando o trabalho e o risco de esquecer algum passo.

Líria

O uso do volume simplifica a implementação da base de dados porque os scripts são executados automaticamente. Além disso, todos os membros da equipa podem utilizar os mesmos ficheiros de inicialização. Se os dados fossem carregados manualmente, cada pessoa teria de repetir o processo de importação, o que poderia gerar diferenças entre os ambientes de trabalho.

3. Se desligarem o computador e voltarem a ligar, os dados permanecem? Testem e documentem o que observam.

R%

Carlos

Após desligar e voltar a ligar o computador, observámos que a base de dados continuou disponível e os dados permaneceram armazenados. Isso acontece porque o Docker guarda as informações do contentor em armazenamento persistente. Quando o contentor é iniciado novamente, os dados continuam acessíveis.

José

No teste realizado, os dados mantiveram-se após reiniciar o computador. As tabelas e os registos continuaram presentes quando o contentor voltou a ser executado. Esse comportamento demonstra que o armazenamento utilizado pelo Docker não é apagado apenas por desligar ou reiniciar a máquina.

Emanuel

Depois de reiniciar o sistema, verificámos que a base de dados ainda continha as mesmas informações inseridas anteriormente. Os contentores foram iniciados novamente e os dados não foram perdidos. Isso confirma que o armazenamento persistente está a funcionar corretamente.

Líria

Durante os testes, desligámos o computador e iniciámos novamente os serviços Docker. Ao aceder à base de dados, constatámos que os dados continuavam disponíveis. Apenas reiniciar ou desligar o computador não elimina as informações armazenadas; elas só seriam removidas caso os volumes ou os dados do contentor fossem apagados explicitamente.

Fase 2 — Arquitectura Distribuída e Fragmentação

2.1 — O que é um Sistema de BD Distribuído?

Uma Base de Dados Distribuída (BDD) é um conjunto de dados logicamente relacionados que estão fisicamente armazenados em múltiplos nós de uma rede. Para o utilizador, parece uma única base de dados — mas por baixo, os dados estão espalhados por vários computadores.

Analogia — O Arquivo da VendaNet Angola

Imaginem que a VendaNet tem um arquivo de facturas. Se tudo estiver numa única pasta em Luanda:

- Se o escritório de Luanda fechar, ninguém acede às facturas — PONTO ÚNICO DE FALHA.
- Se um funcionário de Benguela precisar de uma factura, tem de ligar para Luanda — LATÊNCIA ALTA.

Numa BD Distribuída, cada filial tem os seus próprios dados locais E uma cópia dos dados que precisam das outras filiais. Mais rápido, mais resiliente.

2.2 — Fragmentação Horizontal vs Vertical

Fragmentação é o processo de dividir uma tabela em partes menores (fragmentos) que são armazenados em nós diferentes.

Tipo	Fragmentação Horizontal	Fragmentação Vertical
Definição	Divide as LINHAS da tabela por critério (ex.: por província).	Divide as COLUNAS da tabela (ex.: dados de preço separados dos dados descritivos).
Analogia	Cada filial guarda as facturas dos seus próprios clientes.	O catálogo de produtos separa a ficha técnica do histórico de preços.
Exemplo no MercadoKwanza	Nó-Luanda guarda VENDA WHERE loja_id IN (lojas de Luanda).	Nó-Luanda guarda id, descricao, categoria; outro nó guarda id, preco, activo.

2.3 — Explorar os Fragmentos no MercadoKwanza

Executar os seguintes comandos no nó Luanda para criar e explorar a fragmentação horizontal das vendas:

13. Criar views que simulam os fragmentos locais de cada nó:

```
-- Executar no nó Luanda (porta 3306):
```

```
USE mercadokwanza;

-- Fragmento de vendas da Luanda (lojas com provincia_id = 1)
CREATE OR REPLACE VIEW frag_venda_luanda AS
  SELECT v.* FROM VENDA v
  JOIN LOJA l ON v.loja_id = l.id
  WHERE l.provincia_id = 1;

-- Fragmento de vendas de Benguela (provincia_id = 2)
CREATE OR REPLACE VIEW frag_venda_benguela AS
  SELECT v.* FROM VENDA v
  JOIN LOJA l ON v.loja_id = l.id
  WHERE l.provincia_id = 2;

-- Fragmento de vendas de Huambo (provincia_id = 3)
CREATE OR REPLACE VIEW frag_venda_huambo AS
  SELECT v.* FROM VENDA v
  JOIN LOJA l ON v.loja_id = l.id
  WHERE l.provincia_id = 3;
```

14. Verificar o tamanho de cada fragmento e a completude (soma deve igualar o total):

```
SELECT 'Luanda' AS provincia, COUNT(*) AS total FROM frag_venda_luanda UNION
ALL
SELECT 'Benguela' AS provincia, COUNT(*) AS total FROM frag_venda_benguela
UNION ALL
SELECT 'Huambo' AS provincia, COUNT(*) AS total FROM frag_venda_huambo
UNION ALL
SELECT 'TOTAL GERAL' AS provincia, COUNT(*) AS total FROM VENDA;
```

15. Propor uma fragmentação vertical para a tabela PRODUTO. Escrever o CREATE VIEW correspondente separando os atributos de catálogo (descricao, categoria) dos atributos comerciais (preco, activo).

REGISTO DE RESULTADOS — FASE-2

Total vendas — fragmento Luanda	
Total vendas — fragmento Benguela	
Total vendas — fragmento Huambo	
Soma dos 3 fragmentos = total? (S/N)	
Query de fragmentação vertical escrita pelo grupo? (S/N)	

REFLEXÃO ESCRITA — resposta individual no relatório

- Segundo Özsu & Valduriez (2020), um esquema de fragmentação válido deve respeitar três propriedades: completude, disjunção e reconstituição. As views que criaram respeitam estas três propriedades? Justifiquem para cada uma.

2. Quais são as limitações de simular fragmentação com VIEWS em vez de distribuir fisicamente os dados por diferentes servidores? O que é que as VIEWS não conseguem simular?

3. Num cenário real, que factores decidem se uma tabela deve ser fragmentada horizontalmente, verticalmente, ou não fragmentada de todo?

Fase 3 — Replicação MySQL: Master e Slaves

3.1 — O que é a Replicação e Para que Serve?

Replicação é o processo pelo qual um servidor de BD (o Master) copia automaticamente todas as suas operações de escrita para um ou mais servidores secundários (os Slaves). Os Slaves ficam com uma cópia actualizada dos dados do Master.

Analogia — O Espelho Inteligente

O Master é o artista que pinta um quadro. Os Slaves são espelhos que copiam cada pincelada em tempo real. Se o artista pintar uma árvore, os espelhos mostram a árvore. Se o artista apagar uma linha, os espelhos apagam também.

A vantagem: se alguém quiser apenas ver o quadro (leitura), pode olhar para qualquer espelho sem incomodar o artista. Se quiser alterar o quadro (escrita), só pode falar com o artista.

3.2 — Configurar a Replicação

16. Obter as coordenadas do binary log no Master (Luanda):

```
docker exec -it no-luanda mysql -uroot -pkwanza2024 -e "SHOW MASTER STATUS\G"

-- Anotar os valores:
-- File:      mysql-bin.000001      (ou outro número)
-- Position: 157                    (ou outro valor)
```

17. Configurar o Slave Benguela (repetir para Huambo trocando o container):

```
docker exec -it no-benguela mysql -uroot -pkwanza2024

CHANGE MASTER TO
  MASTER_HOST='no-luanda',
  MASTER_USER='root',
  MASTER_PASSWORD='kwanza2024',
  MASTER_LOG_FILE='mysql-bin.000001',    -- valor anotado no passo anterior
  MASTER_LOG_POS=157;                  -- valor anotado no passo anterior

START SLAVE;
SHOW SLAVE STATUS\G
```

18. Verificar os campos mais importantes do SLAVE STATUS:

```
-- Campos a observar e anotar:
-- Slave_IO_Running:      Yes    ← o Slave está a ler o log do Master
-- Slave_SQL_Running:     Yes    ← o Slave está a executar as operações
-- Seconds_Behind_Master: 0      ← atraso em segundos (0 = sincronizado)
-- Last_Error:            ' '    ← deve estar vazio (sem erros)
```

19. Testar a propagação — inserir dados no Master e verificar nos Slaves:

```
-- No Master (Luanda):
docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadokwanza
```

```

INSERT INTO PRODUTO (descricao, categoria, preco, activo)
VALUES ('Sabão Protex 200g', 'Higiene', 850, 1),
      ('Arroz Precioso 5kg', 'Alimentação', 3500, 1),
      ('Pilha AA Energizer x4', 'Electrónica', 1200, 1);

-- No Slave (Benguela) – verificar propagação:
docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadokwanza
SELECT id, descricao, categoria, preco FROM PRODUTO ORDER BY id DESC LIMIT 5;

```

REGISTO DE RESULTADOS — FASE-3

Slave_IO_Running — Benguela	
Slave_SQL_Running — Benguela	
Seconds_Behind_Master — Benguela	
Slave_IO_Running — Huambo	
Seconds_Behind_Master — Huambo	
Os 3 produtos inseridos aparecem no Slave Benguela? (S/N)	
Os 3 produtos inseridos aparecem no Slave Huambo? (S/N)	

20. Simular uma falha do Master e observar o comportamento:

```

# Pausar o contentor do Master (simula uma falha de rede): docker
pause no-luanda

# Tentar inserir um produto no Master (vai falhar): docker exec -it no-
luanda mysql -uroot -pkwanza2024 mercadokwanza -e "SELECT 1;"

# Tentar fazer uma leitura no Slave Benguela: docker exec -it no-
benguela mysql -uroot -pkwanza2024 mercadokwanza \ -e "SELECT
COUNT(*) FROM PRODUTO;"

# Restaurar o Master: docker
unpause no-luanda

```

REGISTO DE RESULTADOS — FASE-3B

O que aconteceu às leituras no Slave quando o Master estava pausado?	
O que aconteceu quando tentaram escrever com o Master pausado?	
Após restaurar o Master, o Slave sincronizou automaticamente? (S/N)	

REFLEXÃO ESCRITA — resposta individual no relatório

1. A replicação Master-Slave é síncrona ou assíncrona? O que significa isso para a consistência dos dados? Se um cliente comprar um produto em Luanda (Master) e imediatamente consultar o stock em Benguela (Slave), pode ver um valor desactualizado?
2. Qual o papel do binary log (binlog) na replicação? O que aconteceria se o binary log fosse apagado acidentalmente no Master enquanto os Slaves ainda não tinham lido todas as entradas?
3. Numa arquitectura real com 10 Slaves, o que acontece se o Master cair permanentemente? Que estratégia poderia o grupo adoptar para tornar o sistema mais resiliente?

Fase 4 — Transações, ACID e Two-Phase Commit

4.1 — O que é uma Transação?

Uma transação é um conjunto de operações que devem ser executadas como uma unidade: ou todas têm sucesso (COMMIT), ou nenhuma é aplicada (ROLLBACK). Não existe meio-caminho.

Analogia — A Transferência Bancária

Imagine uma transferência de 50 000 Kz da conta A para a conta B:

OPERAÇÃO 1: Debitar 50 000 Kz na conta A.

OPERAÇÃO 2: Creditar 50 000 Kz na conta B.

Se a luz falhar entre as duas operações, o que acontece? Sem transação, o dinheiro desaparece. Com transação, o sistema reverte a operação 1 (ROLLBACK) e ninguém perde dinheiro.

As propriedades que garantem isto chama-se ACID:

Letra	Propriedade	O que significa
A	Atomicidade	Tudo ou nada. Ou todas as operações da transação são aplicadas, ou nenhuma.
C	Consistência	A base de dados passa de um estado válido para outro estado válido. As regras (constraints) são sempre respeitadas.
I	Isolamento	Transações concorrentes não se vêem umas às outras. O resultado é como se tivessem sido executadas em série.
D	Durabilidade	Após o COMMIT, os dados estão guardados permanentemente, mesmo que o sistema falhe a seguir.

4.2 — Transação Local no MySQL

Começar por uma transação simples num único nó para compreender os comandos básicos:

```
docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadokwanza

-- Ver o stock actual de um produto:
SELECT produto_id, loja_id, quantidade FROM STOCK
WHERE produto_id = 1 AND loja_id = 1;

-- Iniciar a transação:
START TRANSACTION;
```

```
-- Vender 5 unidades (decrementar stock):
UPDATE STOCK SET quantidade = quantidade - 5

WHERE produto_id = 1 AND loja_id = 1 AND quantidade >= 5;

-- Verificar se a linha foi afectada:
SELECT ROW_COUNT() AS linhas_afectadas;

-- Verificar o novo valor (ainda não confirmado):
SELECT quantidade FROM STOCK WHERE produto_id = 1 AND loja_id = 1;

-- Simular um erro: forçar ROLLBACK
ROLLBACK;

-- Confirmar que o valor voltou ao original:
SELECT quantidade FROM STOCK WHERE produto_id = 1 AND loja_id = 1;

-- Repetir o START TRANSACTION e desta vez fazer COMMIT: START
TRANSACTION;
UPDATE STOCK SET quantidade = quantidade - 5 WHERE produto_id = 1 AND loja_id
= 1 AND quantidade >= 5;
COMMIT;
SELECT quantidade FROM STOCK WHERE produto_id = 1 AND loja_id = 1;
```

REGISTO DE RESULTADOS — FASE-4A

Quantidade ANTES do UPDATE	
Quantidade APÓS UPDATE (antes do ROLLBACK)	
Quantidade APÓS ROLLBACK (igual ao inicial?)	
Quantidade APÓS COMMIT	

4.3 — Transação Distribuída com Python (Two-Phase Commit Simplificado)

Numa transação distribuída, as operações acontecem em nós diferentes. O Two-Phase Commit (2PC) é o protocolo que garante atomicidade entre nós: primeiro todos confirmam que estão prontos (fase 1 — prepare), depois todos executam o commit (fase 2 — commit).

Instalar o Python e o conector MySQL:

```
python --version # verificar instalação pip
install mysql-connector-python
```

Criar o ficheiro scripts/transacao.py com o seguinte conteúdo:

```
import mysql.connector
```

```
# — Conectar aos dois nós ————— def
conectar(porta):    return mysql.connector.connect(      host='127.0.0.1',
port=porta,        user='root', password='kwanza2024',
```

```

        database='mercadokwanza',          autocommit=False          #
    IMPORTANTE: desactivar autocommit      )

# — Cenário: cliente de Luanda compra produto cujo stock está em Benguela
PRODUTO_ID = 5
LOJA_BENGUELA = 6      # loja com provincia_id = 2
LOJA_LUANDA   = 1      # loja com provincia_id = 1
CLIENTE_ID    = 42
QUANTIDADE    = 10

no_luanda     = conectar(3306) no_benguela
               = conectar(3307)

cur_l = no_luanda.cursor() cur_b
       = no_benguela.cursor()

try:
    # — FASE 1: Verificar e decrementar stock em Benguela —
    cur_b.execute(
        'SELECT quantidade FROM STOCK WHERE produto_id=%s AND loja_id=%s FOR
        UPDATE',
        (PRODUTO_ID, LOJA_BENGUELA)
    )
    resultado = cur_b.fetchone()      if resultado
    is None or resultado[0] < QUANTIDADE:
        raise Exception(f'Stock insuficiente: disponível={resultado[0]} if
        resultado else 0}')

    cur_b.execute(
        'UPDATE STOCK SET quantidade=quantidade-%s WHERE produto_id=%s AND
        loja_id=%s',
        (QUANTIDADE, PRODUTO_ID, LOJA_BENGUELA)
    )
    print(f'[Benguela] Stock decrementado: -{QUANTIDADE} unidades')

    # — FASE 2: Registrar a venda em Luanda —
    cur_l.execute(
        'INSERT INTO VENDA (loja_id, cliente_id, data_venda, total) VALUES
        (%s,%s,NOW(),%s)',
        (LOJA_LUANDA, CLIENTE_ID, QUANTIDADE * 1500) # preço fictício
    )
    venda_id = cur_l.lastrowid

    cur_l.execute(
        'INSERT INTO ITEM_VENDA (venda_id, produto_id, qtd, preco_unit,
        desconto)',
        'VALUES (%s,%s,%s,%s,%s)',
        (venda_id, PRODUTO_ID, QUANTIDADE, 1500, 0.0)
    )
    print(f'[Luanda] Venda registada: id={venda_id}')

    # — COMMIT nos dois nós (2PC simplificado)
    —   no_benguela.commit()      no_luanda.commit()
    print('✓ Transação concluída com sucesso em ambos os nós.')

```

```
except Exception as e:
    # — ROLLBACK nos dois nós —
    no_benguela.rollback()    no_luanda.rollback()
    print(f'X ROLLBACK efectuado em ambos os nós. Motivo: {e}')

finally:
    cur_l.close();    cur_b.close()
    no_luanda.close(); no_benguela.close()
```

21. Executar o script e registar os resultados:

```
python scripts/transacao.py
```

22. Forçar um erro artificial: comentar a linha do INSERT em Luanda antes do commit e executar novamente. Observar o ROLLBACK.

23. Verificar no MySQL que os dados são consistentes após o ROLLBACK.

REGISTO DE RESULTADOS — FASE-4B

Stock em Benguela ANTES da transação	
Stock em Benguela APÓS a transação (sucesso)	
N.º de vendas em Luanda ANTES	
N.º de vendas em Luanda APÓS (sucesso)	
Resultado com erro forçado — ROLLBACK executado? (S/N)	
Stock em Benguela após ROLLBACK (igual ao inicial?)	

REFLEXÃO ESCRITA — resposta individual no relatório

1. O protocolo Two-Phase Commit que implementaram é simplificado — não trata todos os casos de falha. O que aconteceria se o nó Benguela fizesse COMMIT mas a ligação falhasse antes de o nó Luanda fazer o seu COMMIT? Este estado chama-se 'indoubt transaction'. Como poderia ser resolvido?
2. Qual a diferença entre o nível de isolamento READ COMMITTED e SERIALIZABLE? Para um sistema de stock em tempo real, qual escolheriam e porquê?
3. O uso de 'FOR UPDATE' no SELECT em Benguela bloqueou o registo. Que problema resolve este bloqueio? Que problema pode criar se muitas transações tentarem aceder ao mesmo produto em simultâneo?

PARTE 2 — DEFESA EM GRUPO

15 a 21 de Junho de 2026 | Estratégia: 3 Problemas × 2 Grupos cada

Estratégia Pedagógica dos 6 Grupos

Esta fase adopta uma dinâmica de defesa entre pares. Cada problema é resolvido por dois grupos em paralelo: cada grupo trabalha num cenário diferente (A ou B) mas sobre o mesmo conceito. Na defesa, o Grupo Defensor apresenta; o Grupo Observador questiona e critica com base na sua própria implementação do mesmo problema.

Problema	Grupos	Cenário A	Cenário B
Problema 1 Replicação + Fragmentação	G1 (A) G2 (B)	Replicação de STOCK entre nós. G1 adiciona 50 produtos e analisa a propagação temporal.	Replicação de CLIENTE entre nós. G2 regista 100 novos clientes e analisa o lag de replicação.
Problema 2 Transações Distribuídas	G3 (A) G4 (B)	Venda de produto com stock em Benguela para cliente de Luanda — simular falha no COMMIT do nó destino.	Transferência de stock entre lojas de províncias diferentes — simular stock insuficiente e garantir ROLLBACK total.
Problema 3 SQL vs NoSQL + CAP	G5 (A) G6 (B)	Migrar VENDA + ITEM_VENDA para MongoDB. Comparar tempos de agregação (total por loja) entre MySQL e MongoDB.	Migrar CLIENTE + VENDA para MongoDB. Comparar tempos de agregação (clientes com mais compras) entre MySQL e MongoDB.

Protocolo da Sessão de Defesa (cada sessão: 25 minutos por problema)

Etapa	Descrição
1	Grupo Defensor demonstra o sistema ao vivo: cluster em funcionamento, operação do cenário, resultados obtidos. (10 min)
2	Grupo Observador apresenta 3 perguntas técnicas preparadas durante a semana e partilha 2 diferenças que encontrou no seu cenário. (8 min)
3	Grupo Defensor responde e justifica as suas decisões de design. (5 min)
4	Docente pode intervir com questões a qualquer membro de qualquer grupo. (2 min)

Dataset da Parte 2 — novo script com os líderes

Antes da Parte 2, os líderes dos 6 grupos reúnem-se novamente e geram um segundo dataset — mesmo esquema MercadoKwanza, mas com novos dados (nova

semente aleatória no script) para que a Parte 2 parta de dados frescos e a análise não fique contaminada pelos testes da Parte 1.

Prompt sugerido para a IA: 'Usa o mesmo esquema SQL do MercadoKwanza mas gera dados com semente RAND(42) para que sejam diferentes dos anteriores. Volume: 30 000 vendas, 5 000 clientes, 200 produtos.'

Problema 1 — Replicação e Fragmentação (Grupos 1 e 2)

Conceito central

Replicação MySQL Master-Slave + Fragmentação horizontal — observar a propagação de dados entre nós e analisar o comportamento em caso de atraso (lag) de replicação.

Cenário A — Grupo 1 (Defensor na primeira ronda)

O grupo 1 vai simular a expansão do catálogo de produtos do MercadoKwanza. A sede em Luanda (Master) adiciona 50 novos produtos e o grupo mede quanto tempo demora a propagação para os Slaves de Benguela e Huambo.

24. Registrar o estado inicial de PRODUTO em todos os nós:

```
-- Executar em cada nó e anotar:
docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadokwanza -e 'SELECT
COUNT(*) AS total_luanda FROM PRODUTO;' docker exec -it no-benguela mysql -
uroot -pkwanza2024 mercadokwanza -e 'SELECT COUNT(*) AS total_benguela FROM
PRODUTO;' docker exec -it no-huambo mysql -uroot -pkwanza2024 mercadokwanza
-e 'SELECT
COUNT(*) AS total_huambo FROM PRODUTO;'
```

25. Inserir 50 produtos novos no Master usando uma stored procedure ou um loop na aplicação. Pedir à IA um script para inserir 50 produtos com nomes realistas angolanos em categorias variadas.

26. Imediatamente após o INSERT, verificar o COUNT em Benguela. Aguardar 5 segundos e verificar novamente. Registrar o timestamp de cada verificação.

27. Verificar o atraso com SHOW SLAVE STATUS durante a inserção:

```
docker exec -it no-benguela mysql -uroot -pkwanza2024 \
-e 'SHOW SLAVE STATUS\G' | grep -E
'Seconds_Behind|Slave_IO|Slave_SQL|Last_Error'
```

28. Criar as views de fragmentação horizontal de STOCK por província e verificar a distribuição:

```
CREATE OR REPLACE VIEW frag_stock_luanda AS
SELECT s.* FROM STOCK s JOIN LOJA l ON s.loja_id=l.id WHERE
l.provincia_id=1;

CREATE OR REPLACE VIEW frag_stock_benguela AS
SELECT s.* FROM STOCK s JOIN LOJA l ON s.loja_id=l.id WHERE
l.provincia_id=2;

SELECT 'Luanda' AS prov, SUM(quantidade) AS stock_total FROM
frag_stock_luanda
UNION ALL
SELECT 'Benguela' AS prov, SUM(quantidade) AS stock_total FROM
frag_stock_benguela;
```


REGISTO DE RESULTADOS — P1-A

COUNT PRODUTO no Luanda antes dos INSERTs	
COUNT PRODUTO no Benguela antes dos INSERTs	
COUNT PRODUTO no Benguela imediatamente após INSERT	
COUNT PRODUTO no Benguela após 5 segundos	
Seconds_Behind_Master máximo observado	
Stock total fragmento Luanda	
Stock total fragmento Benguela	

REFLEXÃO ESCRITA — resposta individual no relatório

1. O Slave recebeu os dados antes ou depois dos 5 segundos? O que influencia este tempo de propagação?
2. Se neste momento o Master caísse, os Slaves teriam todos os dados? O que poderia ter ficado por replicar?
3. A fragmentação de STOCK por província faz sentido para o negócio do MercadoKwanza? Que consultas seriam mais rápidas e que consultas seriam mais lentas com esta fragmentação?

Cenário B — Grupo 2 (Defensor na segunda ronda)

O grupo 2 vai simular o registo massivo de novos clientes na época de promoções. A sede em Luanda (Master) regista 100 novos clientes e o grupo analisa o comportamento da replicação e implementa fragmentação de CLIENTE por província.

29. Registrar o estado inicial de CLIENTE em todos os nós.
30. Pedir à IA um script SQL ou Python que insira 100 novos clientes com dados angolanos realistas distribuídos pelas 3 províncias (aprox. 50% Luanda, 30% Benguela, 20% Huambo).
31. Medir o tempo de propagação nos Slaves (igual ao Cenário A, mas para CLIENTE).
32. Criar views de fragmentação de CLIENTE por província e comparar o total com o COUNT(*) geral:

```
CREATE OR REPLACE VIEW frag_cliente_luanda AS SELECT * FROM CLIENTE WHERE
provincia_id=1; CREATE OR REPLACE VIEW frag_cliente_benguela AS SELECT * FROM
CLIENTE WHERE provincia_id=2; CREATE OR REPLACE VIEW frag_cliente_huambo AS
SELECT * FROM CLIENTE WHERE provincia_id=3;
```

```
-- Verificar completude:
SELECT
  (SELECT COUNT(*) FROM frag_cliente_luanda) +
  (SELECT COUNT(*) FROM frag_cliente_benguela) +
  (SELECT COUNT(*) FROM frag_cliente_huambo) AS soma_fragmentos,
  (SELECT COUNT(*) FROM CLIENTE) AS total_geral;
```

33. Calcular o faturamento médio por cliente por província para entender o valor de cada fragmento:

```
SELECT p.nome AS provincia, COUNT(DISTINCT c.id) AS clientes,
ROUND(SUM(v.total)/COUNT(DISTINCT c.id),2) AS faturamento_medio_por_cliente
FROM CLIENTE c
JOIN PROVINCIA p ON c.provincia_id = p.id
LEFT JOIN VENDA v ON v.cliente_id = c.id
GROUP BY p.nome ORDER BY faturamento_medio_por_cliente DESC;
```

REGISTO DE RESULTADOS — P1-B

COUNT CLIENTE antes dos INSERTs (Luanda)	
COUNT CLIENTE após INSERTs (Luanda)	
COUNT CLIENTE após 5s (Benguela — propagou?)	
Soma dos 3 fragmentos = total geral? (S/N)	
Província com maior faturamento médio por cliente	
Seconds_Behind_Master máximo observado	

REFLEXÃO ESCRITA — resposta individual no relatório

1. Comparando os dois cenários (PRODUTO vs CLIENTE), a propagação foi mais rápida com 50 ou com 100 registos? Qual a explicação técnica?
2. Para o MercadoKwanza, que tabelas faria mais sentido fragmentar por província e que tabelas deveriam existir replicadas integralmente em todos os nós? Justifiquem com base nos padrões de acesso esperados.
3. A view de fragmentação de CLIENTE que criaram viola ou respeita a propriedade de disjunção? Um cliente pode aparecer em dois fragmentos diferentes? Porquê ou porquê não?

Problema 2 — Transações Distribuídas (Grupos 3 e 4)

Conceito central

Atomicidade em transações que envolvem múltiplos nós. Two-Phase Commit simplificado em Python. Comportamento em caso de falha e ROLLBACK total.

Cenário A — Grupo 3 (Defensor na primeira ronda)

Simular uma venda interfiliar: um cliente de Luanda compra um produto cujo stock existe apenas numa loja de Benguela. A venda é registada em Luanda; o stock é decrementado em Benguela. Simular uma falha no momento do COMMIT em Luanda e garantir o ROLLBACK total.

34. Verificar o stock actual do produto a usar (escolher um produto com stock > 20 em Benguela):

```
docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadokwanza -e \
'SELECT s.produto_id, p.descricao, s.loja_id, l.nome AS loja, s.quantidade
FROM STOCK s JOIN PRODUTO p ON s.produto_id=p.id JOIN LOJA l ON
s.loja_id=l.id
WHERE l.provincia_id=2 AND s.quantidade > 20 LIMIT 5;'
```

35. Adaptar o script transacao.py da Parte 1 para este cenário. Executar com sucesso primeiro.
36. Modificar o script para simular uma falha artificial: após o UPDATE em Benguela e antes do INSERT em Luanda, lançar uma excepção. Verificar que o ROLLBACK é executado em ambos os nós e que o stock volta ao valor original.
37. Adicionar ao script um log que registe: timestamp, nó, operação e resultado (COMMIT ou ROLLBACK):

```
import datetime def log(no, operacao, resultado):      ts =
datetime.datetime.now().strftime('%H:%M:%S.%f')[:12]
print(f'[{ts}] [{no:10}] {operacao:35} -> {resultado}')

# Usar no script:
log('Benguela', 'UPDATE STOCK', 'OK')
log('Luanda', 'INSERT VENDA', 'ERRO - simulado')
log('Benguela', 'ROLLBACK', 'executado') log('Luanda',
'ROLLBACK', 'executado')
```

REGISTO DE RESULTADOS — P2-A

Stock produto em Benguela ANTES	
Stock produto em Benguela APÓS transação OK	
Stock produto em Benguela APÓS ROLLBACK (= ao inicial?)	
N.º vendas em Luanda ANTES	
N.º vendas em Luanda APÓS transação OK	

N.º vendas em Luanda APÓS ROLLBACK (= ao inicial?)	
O log foi correctamente impresso? (S/N)	

REFLEXÃO ESCRITA — resposta individual no relatório

1. O script garante atomicidade total mesmo com a falha simulada? O que aconteceria se a falha ocorresse entre o commit de Benguela e o commit de Luanda — seria possível garantir atomicidade sem um coordenador externo?
2. Qual a diferença entre o ROLLBACK que implementaram e um ROLLBACK automático feito pelo MySQL quando uma query tem erro de sintaxe?
3. Em termos de desempenho, qual é o custo de manter uma transação aberta em dois nós durante um longo período? O que pode acontecer a outros processos que tentem aceder aos mesmos dados?

Cenário B — Grupo 4 (Defensor na segunda ronda)

Simular uma transferência de stock entre lojas de províncias diferentes: retirar 30 unidades da loja de Huambo e adicionar a uma loja de Benguela. Simular o caso em que o stock de Huambo é insuficiente e garantir que nenhuma operação é persistida.

38. Verificar o stock do produto escolhido em Huambo e Benguela:

```
-- Verificar em Huambo: docker exec -it no-huambo mysql -uroot -
pkwanza2024 mercadokwanza -e \
'SELECT s.produto_id, p.descricao, s.loja_id, s.quantidade
FROM STOCK s JOIN PRODUTO p ON s.produto_id=p.id
WHERE s.loja_id IN (SELECT id FROM LOJA WHERE provincia_id=3) LIMIT 5;'
```

39. Criar o script scripts/transferencia.py que:

- Verifica se o stock em Huambo é ≥ 30 .
- Se sim: decrementa em Huambo e incrementa em Benguela (transação distribuída com COMMIT nos dois nós).
- Se não: lança uma excepção e faz ROLLBACK em ambos os nós.

40. Executar primeiro com quantidade suficiente. Depois reduzir o stock manualmente para menos de 30 e executar novamente para observar o ROLLBACK:

```
-- Forçar stock insuficiente para teste:
docker exec -it no-huambo mysql -uroot -pkwanza2024 mercadokwanza -e \
'UPDATE STOCK SET quantidade=10 WHERE produto_id=X AND loja_id=Y;'
```

Executar o script (deve fazer ROLLBACK):

```
python scripts/transferencia.py
```

```
-- Verificar que os valores não mudaram em nenhum nó: docker exec -it no-
huambo mysql -uroot -pkwanza2024 mercadokwanza -e 'SELECT quantidade FROM
STOCK WHERE produto_id=X AND loja_id=Y;'
```

```
docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadokwanza -e 'SELECT
quantidade FROM STOCK WHERE produto_id=X AND loja_id=Z;'
```

REGISTO DE RESULTADOS — P2-B

Stock Huambo ANTES (suficiente)	
Stock Huambo APÓS transferência OK	
Stock Benguela ANTES	
Stock Benguela APÓS transferência OK	
Resultado quando stock insuficiente — ROLLBACK? (S/N)	
Stock Huambo após ROLLBACK (= ao que foi forçado?)	
Stock Benguela após ROLLBACK (= ao inicial?)	

REFLEXÃO ESCRITA — resposta individual no relatório

1. Em que aspecto o Cenário B é mais complexo do que o Cenário A? O facto de envolver incremento E decremento adiciona algum risco de inconsistência que não existe no Cenário A?
2. Comparem os vossos resultados com os do Grupo 3. Os dois cenários demonstram o mesmo princípio (atomicidade)? Que diferenças encontraram na implementação?
3. Se a transferência envolvesse 10 lojas em vez de 2 nós, o protocolo Two-Phase Commit que implementaram ainda seria suficiente? Que protocolo seria necessário?

Problema 3 — SQL Distribuído vs NoSQL e Teorema CAP (Grupos 5 e 6)

Conceito central

Teorema CAP, modelos ACID vs BASE, migração de dados para MongoDB, comparação de desempenho de consultas analíticas entre MySQL e MongoDB.

3.1 — O Teorema CAP

Eric Brewer formulou em 2000 a conjectura que qualquer sistema distribuído só pode garantir simultaneamente duas das três propriedades seguintes:

Letra	Propriedade	Significa...
C	Consistency	Todos os nós vêem exactamente os mesmos dados ao mesmo tempo. Uma leitura retorna sempre o valor mais recente escrito.
A	Availability	O sistema responde sempre a qualquer pedido, mesmo que a resposta não seja a mais recente.
P	Partition Tolerance	O sistema continua a funcionar mesmo que a rede entre nós falhe (partição de rede).

Numa rede real, as falhas de rede (P) são inevitáveis — por isso P é sempre necessário. A escolha real é entre C e A: quando a rede falha, o sistema sacrifica consistência (AP) ou disponibilidade (CP)?

Analogia — O Teorema CAP num banco angolano

Sistema CP: quando a comunicação com a sede em Luanda cai, as agências de Benguela param de aceitar saques até a ligação ser restaurada — preferem parar (sem disponibilidade) a arriscar dados incorrectos.

Sistema AP: as agências de Benguela continuam a aceitar saques mesmo sem comunicação com Luanda — preferem continuar disponíveis e sincronizar os dados depois (eventual consistency).

A escolha depende do negócio. Numa operação de stock do MercadoKwanza, qual faz mais sentido?

Cenário A — Grupo 5 (Defensor na primeira ronda)

Migrar VENDA e ITEM_VENDA para o MongoDB como documentos embutidos. Comparar o tempo de uma agregação analítica (total de vendas por loja) entre MySQL e MongoDB.

41. Instalar a biblioteca pymongo:

```
pip install pymongo
```

42. Criar o script scripts/migracao.py que exporta as vendas do MySQL para o MongoDB:

```

import mysql.connector, pymongo, time

# Conectar ao MySQL (Luanda) e ao MongoDB
mysql_conn = mysql.connector.connect(
    host='127.0.0.1',
    port=3306, user='root', password='kwanza2024',
    database='mercadokwanza'
)
mongo_client =
pymongo.MongoClient('mongodb://localhost:27017/') db =
mongo_client['mercadokwanza'] col = db['vendas']
col.drop() # limpar coleção antes de importar

cur = mysql_conn.cursor(dictionary=True)

# Buscar vendas com itens embutidos
cur.execute('SELECT id, loja_id, cliente_id, data_venda, total FROM VENDA')
vendas = cur.fetchall()

docs = [] for v
in vendas:
    cur.execute(
        'SELECT produto_id, qtd, preco_unit, desconto FROM ITEM_VENDA WHERE
        venda_id=%s',
        (v['id'],)
    )
    v['itens'] = cur.fetchall()
v['data_venda'] = str(v['data_venda'])
docs.append(v)

col.insert_many(docs) print(f'Migrados {len(docs)}
documentos para o MongoDB.') cur.close();
mysql_conn.close(); mongo_client.close()

```

43. Executar a migração e verificar no MongoDB:

```

python scripts/migracao.py

# Verificar no MongoDB: docker exec
-it mongo-kwanza mongosh use
mercadokwanza
db.vendas.countDocuments()
db.vendas.findOne() // ver um documento

```

44. Executar a agregação no MongoDB e medir o tempo:

```

// No mongosh - total de vendas por loja:
const t0 = Date.now(); db.vendas.aggregate([
    { $group: { _id: '$loja_id', total: { $sum: '$total' }, count: { $sum: 1 } } },
    { $sort: { total: -1 } }
]).toArray(); print('Tempo MongoDB (ms):',
Date.now() - t0);

```

45. Executar a query equivalente no MySQL e comparar:

```
-- No MySQL Luanda:
SELECT SQL_NO_CACHE loja_id, SUM(total) AS total, COUNT(*) AS n_vendas
FROM VENDA GROUP BY loja_id ORDER BY total DESC; -- Usar SHOW PROFILES
para medir tempo:
SET profiling=1;
SELECT SQL_NO_CACHE loja_id, SUM(total) AS total FROM VENDA GROUP BY loja_id;
SHOW PROFILES;
```

REGISTO DE RESULTADOS — P3-A

N.º documentos migrados para MongoDB	
Tempo agregação MongoDB — total por loja (ms)	
Tempo query MySQL — total por loja (ms)	
Qual foi mais rápido?	
N.º de lojas no resultado da agregação	
Loja com maior total de vendas (MongoDB)	

REFLEXÃO ESCRITA — resposta individual no relatório

1. O MongoDB foi mais ou menos rápido que o MySQL? A que se deve essa diferença? O modelo de documento embutido (itens dentro da venda) tem alguma vantagem para esta consulta específica?
2. O sistema MySQL que usaram (Master-Slave) é CP ou AP segundo o Teorema CAP? E o MongoDB no modo standalone (um único nó)? Justifiquem.
3. Se o MercadoKwanza precisasse de gerar relatórios analíticos diários (vendas por loja, por produto, por período), que sistema recomendariam: MySQL distribuído ou MongoDB? Porquê?

Cenário B — Grupo 6 (Defensor na segunda ronda)

Migrar CLIENTE e VENDA para o MongoDB como documentos embutidos (clientes com lista de vendas). Comparar o tempo de uma consulta analítica (clientes com maior número de compras) entre MySQL e MongoDB.

46. Criar o script de migração scripts/migracao_clientes.py que exporta clientes com as suas vendas embutidas. Pedir à IA para adaptar o script do Cenário A para este modelo.
47. Verificar que o modelo de documento é coerente:

```
// Um documento deve ter este aspecto:
// { id: 42, nome: 'Ana Domingos', nif: '...',
//   provincia_id: 1, telefone: '...',
//   vendas: [
```



```
//      { id: 101, loja_id: 3, data_venda: '2024-03-15', total: 4500 }, //
{ id: 205, loja_id: 1, data_venda: '2024-06-20', total: 2800 },
//    ]
//  }

db.clientes.aggregate([
  { $project: { nome: 1, n_vendas: { $size: '$vendas' } } },
  { $sort: { n_vendas: -1 } },
  { $limit: 10 }
]).toArray()
```

48. Executar a mesma consulta no MySQL e comparar os tempos:

```
SET profiling=1;
SELECT SQL_NO_CACHE c.id, c.nome, COUNT(v.id) AS n_vendas
FROM CLIENTE c LEFT JOIN VENDA v ON v.cliente_id = c.id
GROUP BY c.id, c.nome ORDER BY n_vendas DESC LIMIT 10;
SHOW PROFILES;
```

49. Posicionar o MySQL distribuído e o MongoDB no Teorema CAP e preencher a tabela comparativa:

Critério	MySQL Master-Slave	MongoDB standalone
Modelo de dados	(preencher)	(preencher)
Transações ACID	(preencher)	(preencher)
Escalabilidade horizontal	(preencher)	(preencher)
Posição no CAP	(preencher)	(preencher)
Consistência dos dados	(preencher)	(preencher)
Velocidade em agregações	(preencher)	(preencher)
Recomendação para MercadoKwanza	(preencher)	(preencher)

REGISTO DE RESULTADOS — P3-B

N.º clientes migrados com vendas embutidas	
Tempo consulta MongoDB — top 10 clientes (ms)	
Tempo consulta MySQL — top 10 clientes (ms)	
Qual foi mais rápido?	
Cliente com mais compras (MySQL)	
Cliente com mais compras (MongoDB — deve ser igual?)	

REFLEXÃO ESCRITA — resposta individual no relatório

1. O modelo 'clientes com vendas embutidas' tem um problema quando um cliente tem centenas de vendas. Qual é? Como resolveria o grupo este problema no MongoDB?
2. Comparando os dois cenários deste problema (VENDA embutido em clientes vs clientes embutidos em VENDA), qual o modelo que favorece mais as consultas analíticas do MercadoKwanza? Porquê?
3. Se a base de dados do MercadoKwanza crescer para 10 províncias, 500 lojas e 5 milhões de clientes, que arquitectura recomendariam: MySQL com replicação, MongoDB com sharding, ou uma arquitectura híbrida? Justifiquem com argumentos técnicos.

Relatório Final e Publicação no GitHub

Estrutura do Relatório

O relatório deve ser enviado por e-mail para judson.paiva@isptec.co.ao até ao último dia da Parte 2 (21 de Junho de 2026). Assunto do e-mail: [BDD II] Lab02 — Grupo X — Situação Y.

#	Secção	Conteúdo esperado
1	Capa	Formato ISPTec: nomes, grupo, cenário, número de turma, data.
2	Parte 1 — Resultados	Todos os carimbos preenchidos das Fases 0 a 4, com prints e logs colados.
3	Parte 1 — Reflexões	Todas as reflexões escritas das Fases 0 a 4 com as próprias palavras.
4	Registo de uso de IA	Para cada uso de IA: ferramenta, prompt, resposta da IA, o que o grupo alterou.
5	Parte 2 — Implementação	Descrição técnica do cenário (A ou B) com evidências e o código comentado.
6	Parte 2 — Análise	Tabela comparativa preenchida, carimbos, análise do Teorema CAP.
7	Parte 2 — Reflexões	Reflexões individuais escritas com as próprias palavras.
8	Dificuldades	O que não correu bem, como foi diagnosticado e resolvido.
9	Conclusão	O que aprenderam — em linguagem simples, sem jargão excessivo.
10	GitHub	Link do repositório com README explicativo.
11	Referências	Mínimo 5 fontes académicas em APA 7.ª edição.

Estrutura do Repositório GitHub

O repositório deve estar público e ter o seguinte conteúdo mínimo:

```

bdd2-lab02-grupo[N] /
├── README.md           ← descrição do projecto, como executar,
conclusões ─┬── dados/
│             ├── mercadokwanza_p1.sql ← dataset da Parte 1
│             ├── mercadokwanza_p2.sql ← dataset da Parte 2
│             └── scripts/
│                 ├── transacao.py      ← script de transação distribuída
│                 ├── migracao.py      ← script de migração para MongoDB |
│                 └── [outros scripts]
├── docker-compose.yml  ← configuração do cluster ─┬──
relatorio/
└── Lab02_Grupo[N].pdf ← relatório em PDF

```

Como criar e publicar o repositório

50. Criar uma conta em <https://github.com> se ainda não tiverem.
51. Criar um repositório público com o nome **bdd2-lab02-grupo[N]**.
52. Inicializar localmente: `git init` → `git add .` → `git commit -m 'Lab02 BDD'` → `git push`.
53. Verificar que os ficheiros estão visíveis em **[https://github.com/\[utilizador\]/bdd2-lab02-grupo\[N\]](https://github.com/[utilizador]/bdd2-lab02-grupo[N])**.
54. Copiar o link e incluir no relatório e no corpo do e-mail.

Rubrica de Avaliação — 20 Valores

Critério	Peso	Indicadores de desempenho
Parte 1 — Carimbos preenchidos e correctos	15%	Todos os carimbos com valores reais. Evidências (prints, logs) integradas no relatório.
Parte 1 — Reflexões escritas	15%	Respostas com as próprias palavras, com fundamentação técnica e referências a autores.
Parte 2 — Implementação técnica do cenário	20%	Código funcional, cluster activo, resultados correctos e verificáveis.
Parte 2 — Análise comparativa	10%	Tabela preenchida com argumentos próprios. Posição no CAP justificada.
Registo de uso de IA	10%	Cada uso de IA identificado com prompt, resposta e o que o grupo alterou.
Relatório e GitHub	10%	Estrutura completa, repositório público com README claro.

Defesa — Apresentação (Defensor)	10%	Cluster ao vivo, todos os membros participam, respostas fundamentadas.
Defesa — Observação crítica (Observador)	10%	3 perguntas técnicas de qualidade, 2 diferenças do cenário próprio, Ficha preenchida.

Referências Bibliográficas

- ÖZSU, M. T.; VALDURIEZ, P. (2020). Principles of Distributed Database Systems. 4.^a ed. Springer. — Referência principal sobre fragmentação, replicação e SGBDD.
- BREWER, E. A. (2000). Towards Robust Distributed Systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC). ACM. — Formulação original do Teorema CAP.
- GILBERT, S.; LYNCH, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, 33(2). — Prova formal do Teorema CAP.
- GRAY, J.; LAMPORT, L. (2006). Consensus on Transaction Commit. ACM Transactions on Database Systems, 31(1), pp. 133–160. — Two-Phase Commit e Paxos.
- KLEPPMANN, M. (2017). Designing Data-Intensive Applications. O'Reilly Media. — Capítulos 5, 6 e 9: replicação, particionamento e transações distribuídas.
- CHODOROW, K. (2019). MongoDB: The Definitive Guide. 3.^a ed. O'Reilly Media. — Modelo de documentos, sharding e agregações.
- COULOURIS, G. et al. (2012). Distributed Systems: Concepts and Design. 5.^a ed. Addison-Wesley. — Fundamentos de sistemas distribuídos e tolerância a falhas.
- MySQL 8.0 Reference Manual. Oracle Corporation, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>
- MongoDB Manual 7.0. MongoDB, Inc., 2024. Disponível em: <https://www.mongodb.com/docs/manual/>

— Bom trabalho, e boas descobertas! —

"Um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode tornar o seu computador inutilizável."

— Leslie Lamport